```
{
   foreach (var row in accounts.Rows)
   {
        var owner = row["owner"]
        var points = row["points"]
        var statusPoints = row["statusPoints "]
        ...
   }
}
```

You can also convert tabular parameters to domain objects or collections of domain objects. For example, you could convert the previous table into a list of `Account` objects like this:

```
[Given(@"the following accounts:")]
public void givenTheFollowingAccounts(Table accounts)
{
   var accounts = table.CreateSet<Account>();
}
```

Convert each row of the table into an Account object.

In a similar way, you could use the table in the `Then` step to compare expected results with actual results:

```
Then the accounts should be the following:
  | owner | points | statusPoints   |
  | Jill  | 140000 | 800            |
  | John  | 10000  | 50             |
```

You could implement this step in the following way:

```
[Then(@"the accounts should be the following ")]
public void givenTheFollowingAccounts(Table expectedAccounts)
{
   actualAccounts = AccountDatabaseHelper.GetRelevantAccounts();
   expectedAccounts.CompareToSet<Account>(actualAccounts);
}
```

Retrieve the actual accounts from the database.

Compare the expected accounts to what you actually obtained.

What you've seen here is just a small overview of some of SpecFlow's more interesting features. SpecFlow is a very complete Gherkin implementation that's tightly integrated into the .NET development environment, which makes it a logical choice for teams using BDD in a .NET context.

6.8 Automating scenarios in JavaScript with Cucumber-JS

Users are increasingly expecting richer and more interactive websites, and as a result, JavaScript is becoming increasingly important in modern web development. If you're writing rich client-side applications with complex business logic, it makes good sense to write BDD scenarios related to this business logic natively in JavaScript.

Unit testing is well supported in JavaScript, and low-level BDD unit-testing libraries like Jasmine and Mocha are widely used. We'll look at these libraries in chapter 10, when we discuss BDD-style unit and integration testing. But in this chapter we'll